Le codage des flottants

I) REPRESENTATION DES NOMBRES A VIRGULE :

En notation décimale, on utilise la notation scientifique pour représenter un nombre décimal : $a \times 10^p$
avec $1 \leq a < 10$ et $p \in Z$.
Par exemple : $52980480 = 5{,}2980480 \times 10^7$
De manière analogue, nous allons utiliser cette représentation en base 2 : $s \times m \times 2^n$
où s est le signe du nombre : + ou –, n son exposant : $n \in Z$ et m sa mantisse : $1 \leq m < 2$.

Un nombre flottant est formé de trois éléments : la mantisse, l'exposant et le signe. Le bit de poids fort est le bit de signe : si ce bit est à 1, le nombre est négatif, et s'il est à 0, le nombre est positif. Les e bits suivants représentent l'exposant décalé, et les m bits suivants (m bits de poids faibles) représentent la mantisse.

1°) La norme IEEE 754 :

Format général flottants IEEE 754 :

Signe	**Exposant décalé**	**Mantisse**
(1 bit)	(e bits)	(m bits)

2°) Décalage de l'exposant :

L'exposant peut être positif ou négatif. Afin de garantir la compatibilité des comparaisons entre les entiers naturels et entre les flottants on n'utilise pas le C2. Ici l'exposant est **décalé**, afin de le stocker sous forme d'un nombre non signé. Ce décalage est de $2^{e-1}-1$. (*e* représente le nombre de bits de l'exposant) ; il s'agit donc d'une valeur constante une fois que le nombre de bits *e* est fixé.
L'interprétation d'un nombre (autre qu'infini) est donc : $\boxed{\text{valeur} = \text{signe} \times \text{mantisse} \times 2^{(\text{exposant} - \text{décalage})}}$
Avec signe = ±1, et décalage = $2^{e-1}-1$.

3°) Exceptions :

Dans le cas général (l'exposant est différent de 0 et de 2e-1), le bit de poids fort de la mantisse est toujours égal à 1. De ce fait il est inutile de le représenter explicitement. Le nombre est dit « normalisé ». Si l'exposant décalé est nul, le bit de poids fort de la mantisse est nul, et le nombre est 'dé-normalisé'.
Il y a trois cas particuliers :

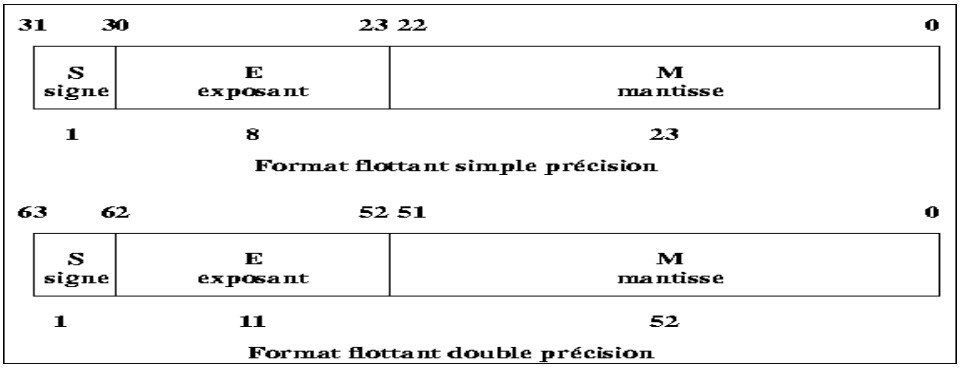
- si l'exposant décalé et la mantisse sont tous deux nuls, le nombre est ±0 (selon le bit de signe).
- si l'exposant décalé est égal à 2e-1, et si la mantisse est nulle, le nombre est ±inf , (selon le bit de signe).
- si l'exposant décalé est égal à 2e-1, mais que la mantisse n'est pas nulle, le nombre est NaN (not a number : pas un nombre).

Type	Exposant décalé	Mantisse
Zéros	0	0
Nombres dénormalisés	0	différente de 0
Nombres normalisés	1 à $2^e - 2$	quelconque
Infinis	$2^e - 1$	0
NaNs	$2^e - 1$	différente de 0

La norme IEEE 754-2008 prévoit 3 types de configurations en fonction du nombre de bits disponibles pour représenter les flottants :

	Simple précision	Double précision	Quadruple précision
Bit de signe	1	1	1
Bits de l'exposant	8	11	15
Bits de la mantisse	23	52	112
Nombre total de bits	32	64	128
Codage de l'exposant	Excédant 127	Excédant 1023	Excédant 16383

Variation de l'exposant	-126 à +127	-1022 à +1023	-16382 à 16383
Plus petit nombre normalisé	2^{-126}	2^{-1022}	2^{-16382}
Plus grand nombre normalisé	2^{+128}	2^{+1024}	2^{+16384}
Echelle des nombres décimaux	Environ 10^{-38} à 10^{+38}	Environ 10^{-308} à 10^{+308}	Environ 10^{-4932} à 10^{+4932}



Exemple :
1 = ($signe, exposant, mantisse$) = (+, 0, 1.0), ce qui donne : 1 = (0, 0 + 2^{e-1} −1, 0)
et en binaire si on représente tous les bits sur 64 bits:
1 = (0, 01111111111, 0000000000000000000000000000000000000000000000000000)
Ce qui est concaténé sur 64 bits :
0011111111110000000000000000000000000000000000000000000000000000
Mais c'est un brin illisible donc on représente cela sous forme hexadécimale :
$3FF0000000000000$

Autres exemples :

Base 10	Base 16	Base 2
3.14	40091eb851eb851f	0100000000001001000111101011100001010001111010111000010100011111

Un exemple :
Quel est le nombre à virgule en décimal représenté par le mot ci-dessous ?

$$\underset{\text{signe}}{1}\ \underbrace{10001000110}_{\text{exposant}}\underbrace{1001001111000011100000000000000000000000000000000000}_{\text{mantisse}}$$

Le signe du nombre est donc $-$.
Le nombre 100 0100 0110 est égal à 1 094 et l'exposant du nombre est n = *1094 - 1023 = 71 (car* $1094 \geq 2^{11-1}$

Sa mantisse est :
$m = \boxed{1}$. 1001 0011 1100 0011 1000 0000 0000 0000 0000 0000 0000 0000 0000

le chiffre devant la virgule

Comment lire convertir en décimal les chiffres après la virgule ?

Prenons exemple sur la notation décimale : 1,023507 s'écrit $1+\frac{2}{10^2}+\frac{3}{10^3}+\frac{5}{10^4}+\frac{7}{10^6}$

symbole, base, rang

Sur ce modèle cela donne :
$m = \boxed{1}$. 1001 0011 1100 0011 1000 0000 0000 0000 0000 0000 0000 0000 0000

$$= 1+\frac{1}{2}+\frac{1}{2^4}+\frac{1}{2^7}+\frac{1}{2^8}+\frac{1}{2^9}+\frac{1}{2^{10}}+\frac{1}{2^{15}}+\frac{1}{2^{16}}+\frac{1}{2^{17}}$$

$$= \frac{206727}{131072}$$

Le nombre représenté est donc $-\frac{206727}{131072}\times 2^{71} = -3{,}724\ldots\ldots \times 10^{21}$

En résumé, quand on utilise 64 bits pour représenter un nombre à virgule (**ce qui est le cas des flottants codés sous Python**), on utilise donc 1 bit pour le signe, 11 bits pour l'exposant et 52 bits pour la mantisse.
• Le signe + est représenté par 0 et le signe − par 1.
• L'exposant *n* est un entier relatif compris entre −1022 et 1023 (puisque codé sur 11 bits). Il doit être représenté comme un entier naturel. Pour cela on utilise un décalage de 1023 plutôt que le complément à deux (qui rendrait la comparaison entre les nombres flottants un peu plus difficile). L'exposant décalé possède donc une valeur comprise entre 1 et 2046. Les deux entiers 0 et 2047 sont réservés pour des situations exceptionnelles (∞, −∞, NaN (Not a Number), etc. . .).
• La mantisse *m* est un nombre binaire à virgule compris entre 1 inclus et 2 exclu, comprenant 52 chiffres après la virgule. Comme cette mantisse est comprise entre 1 et 2, elle a toujours un seul chiffre avant la virgule et ce chiffre est toujours un 1 ; il est donc inutile de le représenter et on utilise les 52 bits pour représenter les chiffres après la virgule.

1°) Est-il nécessaire d'utiliser les 52 bits pour coder toute la mantisse ? Justifier.
2°) Trouver le nombre à virgule représenté par le mot :
$(0\,0\,0\,1\,0\,0\,0\,0\,0\,0\,1\,1\,1\,1\,0\,1\,0\,0\,1\,1\,1\,0\,0\,1\,0\,1\,0\,1\,1\,0\,0\,0\,0\,0\,0

```
from struct import pack,unpack

def floattohexa2(f):
    """Retourne une chaine représentant le flottant en hexadécimal"""
    t=pack('!d',f)
    s=""
    for i in range(0,8):
        s+="{:{fill}8b}".format(t[i], fill='0')
    return s

def triplet(f):
    """
        Prend en argument un flottant en double précision et retourne le triplet:
        -s, le bit de signe
        -e, l'exposant sous forme d'entier
        -m, la mantisse en hexadécimal.
    """
    bytes=pack('!d',f)

    s=floattohexa2(f)
    return (int(s[:1],2),int(s[1:12],2)-1023,"{:x}".format(int(s[12:],2)))

print(triplet(13019830.68))
print(triplet(1.0))
print(triplet(3.14))
```

Résultat :

```
(0, 23, '8d556d5c28f5c')
(0, 0, '0')
(0, 1, '91eb851eb851f')
```

b) Interpréter le résultat obtenu pour 1.0.

2°) <u>Conversion inverse (triplet (s, e, m) → float)</u> :

On donne la fonction inverse dans le cas des flottants normalisés. Elle prend en entrée un triplet (s, e, m) et retourne le flottant correspondant. Il est possible de le faire en une seule expression. L'essayer avec les triplets suivants correspondants aux trois nombres testés précédemment (13019830.68, 1.0, et 3.14)

- (0, 23, '8d556d5c28f5c')
- (0, 0, '0')
- (0, 1, '91eb851eb851f')

```
from decimal import Decimal

def flottant(s,e,m):
    """Retourne le flottant en double précision décrit par :
        -son bit de signe s
        -son exposant e
        -sa mantisse m
    """

    return ((-1)**(s))*(1+int(m,16)/(2**(52)))*2**(e)


print(Decimal(flottant(0, 23, '8d556d5c28f5c')))
print(Decimal(flottant(0, 0, '0')))
print(Decimal(flottant(0, 1, '91eb851eb851f')))
```

a) Quels résultats obtient-on ?

b) Commenter ces résultats.

III) APPLICATION :

A) LA SITUATION ETUDIEE :

SITUATION : ÉCHEC MISSILE PATRIOT A DHARAN EN 1991

Le 25 février 1991, durant la première guerre du Golfe, une batterie de missile [missile Patriot Américain] basée à Dharan, en Arabie Saoudite, échoua à suivre et intercepter un missile Scud irakien en approche. Le missile Scud frappa un baraquement américain, tuant 28 soldats et blessa environ 100 autres personnes. […] Il est apparu que la cause était un mauvais calcul du temps écoulé depuis le démarrage du missile dû à des erreurs d'arrondies numériques. Plus spécifiquement, le temps mesuré était en dixième de secondes par l'horloge interne, puis ensuite multiplié par la valeur 1/10 pour avoir le résultat en secondes. Ce calcul était réalisé sur un registre 24 bits en arithmétique fixe.

La batterie de missile Patriot était allumée depuis 100 heures, et un calcul rapide montre que l'erreur d'arrondi amplifiée était de l'ordre de 0,34 secondes. […] Un missile Scud se déplace à environ 1 676 mètres par seconde, il traverse donc plus d'un demi-kilomètre en 0,34 seconde. Cette distance est suffisante pour que le missile Scud soit hors de la "fenêtre d'interception" que le missile Patriot poursuivait.

Source : https://www.numalis.com/portail-fr-5.php

Objectif : Comprendre ce qui s'est passé le 25 février 1991 à Dharan.

1°) Question préliminaire : codage d'un nombre à virgule par le codage à virgule fixe.
Consulter la vidéo, jusqu'à 5 min 20, disponible à l'adresse suivante :
https://www.youtube.com/watch?v=bDkR-OQiWx0.
En utilisant cette méthode de codage, appelée codage à virgule fixe, coder le nombre $(12{,}125)_{10}$.

2°) Réponse au problème :

a) Coder sur 24 bits en virgule fixe le nombre $(0{,}1)_{10}$ exprimé en base 10.

b) Déterminer la valeur décimale du nombre codé en binaire à virgule fixe par :
$(0{,}000110011001100110011001100)_2$

c) En utilisant les résultats précédents, expliquer d'où proviennent les informations « demi-kilomètre » et « 0,34 seconde » évoquées dans la phrase « il traverse donc plus d'un demi-kilomètre en 0,34 seconde » de l'article.

3°) Synthèse :
On peut remarquer qu'en informatique, à cause du nombre limité de bits, certains nombres décimaux ne peuvent s'exprimer de manière exacte.
Seriez-vous capable de proposer une méthode pour corriger le problème observé ?

B) ET AVEC PYTHON ?

Python code par défaut les nombres à virgules sur 64 bits. Mais comment …

1°) Première observation :

a) Rédiger ci-dessous un algorithme permettant d'afficher les résultats successifs des sommes :

$$0 + 0{,}1 \; ; \; 0{,}1 + 0{,}1 \; ; \; 0{,}2 + 0{,}1 \; ; \; 0{,}3 + 0{,}1 \; ; \; \dots \; ; \; 0{,}9 + 0{,}1 \; ;$$

b) Implémenter cet algorithme en Python. Qu'observe-t-on ?

2°) Essayons d'expliquer cette observation :

a) Compléter le tableau de codage ci-dessous en codant sur 64 bits.

Aide : en binaire : 1 + 0 = 1, 0 + 1 = 1, 1 + 1 = 0 avec une retenue R = 1 (ce qui fait bien $(2)_{10}$, puisque $(10)_2 = (2)_{10}$) pour la colonne à gauche, et 1 + 1 + 1 = 1 avec une retenue R = 1 (ce qui fait bien $(3)_{10}$, puisque $(11)_2 = (3)_{10}$). On commence l'addition par la colonne la plus à droite.
Il est donc conseillé pour remplir la troisième ligne du tableau de faire un tableau du style (64 colonnes pour 64 bits) :

R									
$(0,1)_{10}$	0,	0	0	0	1	1	0	0	…
$(0,2)_{10}$	0,	0	0	1	1	0	0	1	…
$(0,1)_{10} + (0,2)_{10}$									

Codage binaire de $(0,1)_{10}$	
Codage binaire de $(0,2)_{10}$	
Calcul en binaire de $(0,1)_{10} + (0,2)_{10}$	
Codage binaire de $(0,3)_{10}$	

b) Quel est l'écart entre la valeur de $0,1 + 0,2$ attendue et la valeur obtenue par le codage ?
c) Est-ce que le résultat de $0,1 + 0,2$ obtenu à la main est celui donné par Python à l'aide de l'algorithme de la question **1.b** ? Que peut-on en déduire sur la méthode codage des nombres à virgule par Python ?

d) Si le logiciel du missile Patriot avait travaillé sur 64 bits avec virgule fixe, aurait-il atteint le Scud ?

C) <u>EN REALITE</u> :

Python ne représente pas les nombres à virgule en binaire avec la méthode de la virgule fixe. La norme utilisée est la norme appelée **IEEE 754** (vue précédemment) qui est l'analogue en binaire de la notion scientifique.

RAPPEL : Notation scientifique d'un nombre

L'écriture scientifique du nombre $234,5 \times 10^4$ est $2,345 \times 10^6$ car $234,5 \times 10^4 = 2,345 \times 10^2 \times 10^4 = 2,345 \times 10^6$.
D'une manière plus générale, l'écriture scientifique d'un nombre est de la forme $\pm a \times 10^b$ où $1 \leq a < 10$.

Exemples : Donner l'écriture scientifique des nombres

- $30,456 \times 10^3$:
- $30,456 \times 10^{-2}$:
- $0,0345 \times 10^{-2}$:

Préambule : l'écriture au format IEEE 754. Vers une nouvelle représentation.

1°) Représenter le nombre $(4,5625)_{10}$ en binaire à virgule fixe.
2°) Écrire la décomposition en puissances de 2 du nombre $(4,5625)_{10}$.
3°) Par quelle quantité doit-on multiplier $(10,01001)_2$ pour obtenir $(100,1001)_2$?
4°) Même question pour $(1,001001)_2$
5°) Donner la représentation binaire de $(4,5625)_{10}$ et $(-4,5625)_{10}$ en respectant la norme IEEE 754 sur 32 bits.

Retour à $0,1 + 0,2 = 0,30000000000000004$.

1°) Donner la représentation binaire de $(0,1)_{10}$ en IEEE 754 sur 64 bits.
Vérifier votre résultat en consultant le site http://weitz.de/ieee/.
Proposer une explication à la différence constatée.
2°) Donner la représentation binaire de $(0,2)_{10}$ et $(0,3)_{10}$ en IEEE 754 sur 64 bits.
3°) En utilisant le calculateur en ligne, vérifier que l'on retrouve bien le résultat
$0,1 + 0,2 = 0,30000000000000004$ observé avec Python. Commenter.

4°) <u>Bonus : question difficile</u> : en utilisant l'écriture $(0,1)_{10} = (1,1001\ 1001\ \ldots\ 1010)_2 \times 2^{-4}$ proposer une méthode permettant de calculer $(0,1)_{10} + (0,2)_{10}$ lorsque les nombres sont représentés en IEEE 754 sur 64 bits (comme Python).
<u>Aide</u> : écrire les deux nombres avec la puissance de 2 la plus petite des deux.

Expliquer alors le résultat $0,1 + 0,2 = 0,30000000000000004$ observé avec Python.